

BAŞLIK: REST MİMARİSİ VE API TASARIM PRENSİPLERİ

TÜR: Akademik/Teknik Detaylı Anlatım

BÖLÜM 1: REST NEDİR? ROY FIELDING'İN 6 KISITLAMASI

REST (Representational State Transfer), 2000 yılında Roy Fielding'in doktora tezinde tanımladığı, dağıtık sistemler için bir MİMARİ STİLDİR (bir protokol veya standart DEĞİLDİR — bu önemli bir ayrımıdır). Bugün "REST API" dediğimizde genellikle HTTP + JSON kullanan, kaynak tabanlı API'leri kastediyoruz, ama teknik olarak REST'in tanımı 6 kısıtlamaya (constraint) uyan HERHANGİ bir mimariyi kapsar:

- İstemci-Sunucu (Client-Server): İstemci (arayüz) ve sunucu (veri/iş mantığı) birbirinden ayrılmıştır; bağımsız gelişebilirler.
- Durumsuzluk (Statelessness): Her istek, sunucuda önceki isteklere dair hiçbir varsayılan bilgiye dayanmadan, kendi başına anlaşılabilir olmalıdır (bkz. 22.2, Bölüm 1.1).
- Önbelleklenebilirlik (Cacheability): Yanıtlar, önbelleklenip önbelleklenemeyeceklerini açıkça belirtmelidir (Cache-Control başlığı).
- Katmanlı Sistem (Layered System): İstemci, doğrudan asıl sunucuyla mı yoksa aradaki bir proxy/gateway ile mi konuştuğunu bilmek zorunda değildir (bkz. 22.9, API Gateway).
- Tek Tip Arayüz (Uniform Interface): Tüm kaynaklara aynı, standart yöntemlerle erişilir (bu kısıtlama kendi içinde 4 alt-kısıtlamaya ayrılır: kaynak tanımlama, temsil aracılığıyla kaynak manipülasyonu, kendini tanımlayan mesajlar, HATEOAS).
- İsteğe Bağlı Kod-Talep-Üzerine (Code-on-Demand): Sunucu, istemciye çalıştırılabilir kod gönderebilir (örn. JavaScript) — tek isteğe bağlı olan kısıtlamadır.

Pratikte, çoğu "REST API" olarak adlandırılan sistem bu 6 kısıtlamanın tamamına uymaz (özellikle HATEOAS neredeyse hiç uygulanmaz) — buna literatürde bazen alaycı bir şekilde "REST-ish" ya da "HTTP API" denir. Ama pratik mühendislikte önemli olan dogmatik saflık değil, bu prensiplerin getirdiği FAYDALARDIR: öngörülebilirlik, ölçeklenebilirlik ve basitlik.

BÖLÜM 2: RICHARDSON OLGUNLUK MODELİ

Leonard Richardson tarafından geliştirilen bu model, bir API'nin ne kadar "REST'çi" olduğunu 4 seviyede (0-3) sınıflandıran pratik bir araçtır. Çoğu gerçek dünya API'si Seviye 2'de durur — bu genellikle yeterlidir.

Richardson Olgunluk Modeli (REST API Olgunluk Seviyeleri)

SEVIYE 3 — HATEOAS

Yanıtlar, sonraki olası eylemlerin linklerini de içerir.

SEVIYE 2 — HTTP Fiilleri

GET/POST/PUT/DELETE doğru semantiğiyle kullanılır.

SEVIYE 1 — Kaynaklar (Resources)

Her varlık için ayrı URI (/kullanici/42).

SEVIYE 0 — Tek Endpoint

Tek bir URL; işlemler POST gövdesinde belirtilir (RPC tarzı).

Her seviye, bir önceki seviyenin tüm özelliklerini içerir + yeni bir kısıtlama ekler.

Şekil 1 — Richardson Olgunluk Modeli: her seviye bir öncekinin üzerine inşa edilir.

- Seviye 0 (Bataklık / The Swamp of POX): Tek bir URL, tek bir metod (genellikle POST). Hangi işlemin yapılacağı gövde içinde belirtilir — bu aslında REST değil, HTTP üzerinden RPC (Remote Procedure Call) yapmaktır. Eski SOAP servisleri bu modele yakındır.
- Seviye 1 (Kaynaklar): Artık her varlık türü için ayrı bir URL vardır (/kullanici/42, /urunler) ama hala genellikle tek bir HTTP metodu kullanılır.
- Seviye 2 (HTTP Fiilleri): GET/POST/PUT/DELETE doğru anlamlarıyla kullanılır. Endüstri standardının çoğunluğu buradadır — GitHub API, Stripe API gibi büyük API'ler Seviye 2'dedir.
- Seviye 3 (HATEOAS — Hypermedia as the Engine of Application State): Yanıtlar, sadece veriyi değil, bir sonraki olası eylemlerin bağlantılarını (link) da içerir. Örnek: bir sipariş kaydını döndüren yanıt, aynı zamanda {'iptal_et': '/siparisler/42/iptal'} gibi bir link de içerebilir. Teoride istemci, API'yi "keşfederek" kullanabilir hale gelir; pratikte karmaşıklığı nedeniyle nadiren tam olarak uygulanır.

BÖLÜM 3: KAYNAK MODELLEME VE URI TASARIM KURALLARI

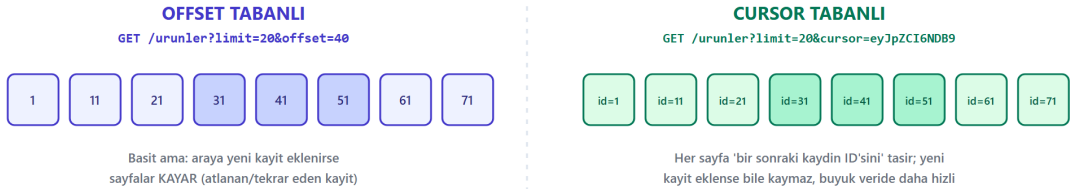
REST'in kalbi "kaynak" (resource) kavramıdır: bir kullanıcı, bir ürün, bir sipariş — hepsi birer kaynaktır ve her kaynağın benzersiz bir adresi (URI) olmalıdır. İyi URI tasarımı için yıllar içinde endüstride yerleşmiş kurallar:

- İsimler kullanın, fiiller değil: **DOĞRU**: GET /urunler — **YANLIŞ**: GET /urunleriGetir. Fiil zaten HTTP metodunda var (GET); URI'de tekrar etmeyin.
- Çoğul isimler kullanın: /urunler (çoğul), /urun değil — koleksiyon/tekil tutarlılığı sağlar.
- İç içe kaynaklarla hiyerarşiyi yansıtın: GET /kullanici/42/siparisler — "42 numaralı kullanıcının siparişleri". Ama 2 seviyeden fazla derinleşmekten kaçının (okunabilirlik azalır).
- Küçük harf ve tire (kebab-case) kullanın: /urun-kategorileri, /urunKategorileri değil.
- Filtreleme, sıralama, arama için query string kullanın: GET /urunler?kategori=elektronik&siralama=fiyat_artan — bunlar URI'nin "kimliğinin" değil, bir SORGUNUN parçasıdır.
- Sürüm bilgisini baştan planlayın: /api/v1/urunler (bkz. Bölüm 5).

BÖLÜM 4: SAYFALAMA (PAGINATION), FİLTRELEME VE SIRALAMA

Bir API asla "tüm kayıtları" tek seferde döndürmemelidir — bir veritabanında 10 milyon satır varsa, bunların hepsini tek bir JSON yanıtında göndermeye çalışmak hem sunucuyu hem istemciyi çöktürür. Bu yüzden büyük koleksiyonlar SAYFALANIR (paginate edilir). İki ana strateji vardır:

Sayfalama (Pagination): Offset vs Cursor Tabanlı



Şekil 2 — Offset tabanlı sayfalama basittir ama büyük/değişken veri setlerinde tutarsızlığa yol açabilir; cursor tabanlı sayfalama bu sorunu çözer.

- Offset Tabanlı: ?limit=20&offset=40 — "40. kayıttan itibaren 20 kayıt getir". Anlaşılması kolaydır ("3. sayfaya git" gibi bir arayüzle doğrudan eşleşir) ama ciddi bir soruna sahiptir: siz 2. sayfayı görüntülerken başka bir kullanıcı 1. sayfaya yeni bir kayıt eklerse, TÜM sonraki sayfalar bir kayıt kayar — bazı kayıtları iki kez görebilir ya da hiç görmeyebilirsiniz. Ayrıca çok büyük offset değerlerinde (örn. offset=1000000) veritabanı performansı ciddi şekilde düşer (veritabanı yine de baştan 1 milyon satırı "saymak" zorundadır).

- Cursor (İmleç) Tabanlı: ?limit=20&cursor=eyJpZCI6NDh9 — cursor, "bir sonraki sayfanın nereden başlayacağını" (genellikle son görülen kaydın ID'si, kodlanmış olarak) taşıyan opak bir imleçtir. Araya yeni kayıt eklense bile kayma olmaz ve büyük veri setlerinde çok daha performanslıdır. Twitter/X, Facebook, Stripe gibi büyük ölçekli API'ler cursor tabanlı sayfalama kullanır. Dezavantajı: "doğrudan 5. sayfaya git" gibi bir arayüz kurmak zordur (sadece ileri/geri gidilebilir).

BÖLÜM 5: API SÜRÜMLEME (VERSIONING) STRATEJİLERİ

Bir API canlıya alındıktan sonra, onu kullanan onlarca/yüzlerce istemci olabilir. API'de GERİYE UYUMSUZ (breaking) bir değişiklik yapmak (bir alanı silmek, bir alanın tipini değiştirmek) tüm o istemcileri aynı anda bozar. Sürümleme, eski davranışı koruyarak yeni bir sürüm sunmanın yoludur:

1. URI Sürümleme: `/api/v1/urunler` vs `/api/v2/urunler`. En yaygın ve en açık yöntemdir; tarayıcıda doğrudan test edilebilir olması büyük bir avantajdır.
2. Header Sürümleme: URL sabit kalır, sürüm bir başlıkla belirtilir (Accept: `application/vnd.sirket.v2+json`). URI'yi "temiz" tutar ama test etmesi ve dokümanete etmesi daha zordur.
3. Query Parametre Sürümleme: `/urunler?version=2`. Nadiren tercih edilir, sürümü "kaynağın kimliğinin" değil bir seçeneğin parçası gibi gösterir.

Endüstri pratiğinde en yaygın ve önerilen yaklaşım URI sürümlemedir — açıklığı ve test kolaylığı nedeniyle bu repository'deki 22.6 (FastAPI) projesinde de bu yaklaşımı kullanacağız.

SONUÇ

REST mimarisinin temel kısıtlamalarını, Richardson Olgunluk Modeli ile bir API'nin olgunluğunu nasıl değerlendireceğimizi, iyi URI tasarım kurallarını, sayfalama stratejilerini ve sürümleme yaklaşımlarını öğrendik. Bir sonraki dosyada (22.4.1), tasarladığımız bu API'yi kötü niyetli kullanıcılardan nasıl KORUYACAĞIMIZI — kimlik doğrulama, yetkilendirme ve hız sınırlama konularını — inceleyeceğiz.